**Universidad Internacional de La Rioja (UNIR)**

Cloud Computing, DevOps y DevOps Culture (Computación en la Nube y Cultura DevOps)

# Actividad: Planificación de un DevOps Dojo

*DevOps Dojo orientado a la escalabilidad: de 10 servidores fijos a Kubernetes con autoescalado para una aplicación Laravel con MongoDB*

---

**Alumno:** Miguel Ángel Guzmán Pérez

**Profesor:** Alberto López Cardoza

**Fecha de entrega:** 2 de noviembre de 2026

# Índice

**1. Introducción**..... 1

1.1. Contexto de la organización y reto del Dojo..... 1

**2. Ejercicio 1. Objetivos del DevOps Dojo**..... 2

**3. Ejercicio 1. Beneficios esperados**..... 3

**4. Ejercicio 2. Alcance del DevOps Dojo**..... 3

**5. Ejercicio 2. Equipos de trabajo participantes**..... 4

**6. Ejercicio 3. Cronograma, sesiones, recursos y tecnologías**..... 5

**7. Ejercicio 4. Métricas de desempeño**..... 7

**8. Ejercicio 4. Sistema de retroalimentación**..... 8

**9. Conclusiones**..... 10

**10. Referencias bibliográficas**..... 10

# 1. Introducción

La cultura DevOps busca que los equipos de desarrollo, operaciones, bases de datos, calidad y seguridad compartan la responsabilidad del servicio y mejoren continuamente mediante la colaboración, la automatización y la medición (Kim et al., 2016). Un **DevOps Dojo** es un entorno de aprendizaje inmersivo en el que los equipos adquieren estas competencias trabajando sobre un **reto real de su propio producto**, guiados por mentores y sin poner en riesgo la producción (Microsoft, 2021, s. f.).

Este documento planifica un Dojo de 12 semanas para una organización ficticia cuya aplicación se cae al superar los 5 000 usuarios concurrentes. No es un proyecto de migración tradicional: el equipo aprende a diagnosticar, optimizar y operar la aplicación en Kubernetes, de modo que la solución y las competencias permanezcan en la organización (DORA, s. f.; Forsgren et al., 2018).

## 1.1. Contexto de la organización y reto del Dojo

**NovaSoft Soluciones** (organización ficticia, 120 empleados) opera un portal de autoservicio para clientes desarrollado en **Laravel**, desplegado en **10 servidores virtuales** (4 vCPU y 8 GB cada uno) con PHP-FPM detrás de un **balanceador Nginx**, y con una base de datos **MongoDB en replica set** de tres nodos (MongoDB, s. f.). En los picos de quincena y fin de mes el portal alcanza unos **5 000 usuarios concurrentes**; por encima de ese umbral los servidores se saturan de CPU, el nodo primario de MongoDB agota su memoria y se reinicia, y el servicio se interrumpe. El **objetivo estratégico para 2027** es soportar una campaña comercial que duplicará la demanda (10 000 usuarios concurrentes) con una disponibilidad del 99,9 %.

| Indicador en el pico (línea base preliminar) | Valor | Causa probable observada |
|---|---|---|
| Punto de quiebre de la aplicación | ≈ 5 000 usuarios | Capacidad fija: 10 nodos que no pueden crecer de forma automática. |
| CPU de los nodos Laravel | 92–98 % | Consultas repetidas sin caché y sesiones gestionadas en disco local. |
| Latencia p95 / errores 5xx | 1 900 ms / 12 % | Procesos PHP-FPM bloqueados esperando respuesta de MongoDB. |
| Conexiones al primario de MongoDB | ≈ 1 800 | Cada proceso PHP-FPM abre conexiones propias; todas las lecturas van al primario. |
| Caídas del servicio / tiempo de recuperación | 6 al mes / 95 min | Sin alertas tempranas; recuperación manual nodo por nodo. |
| Tiempo para añadir capacidad | 2 días | Alta manual de servidores y reconfiguración de Nginx. |

| Indicador en el pico (línea base preliminar) | Valor | Causa probable observada |
|---|---|---|
| Tiempo de despliegue | 120 min | Despliegue manual en 10 servidores con 10 min de indisponibilidad. |

*Tabla 1. Situación inicial de NovaSoft Soluciones (datos ficticios).*

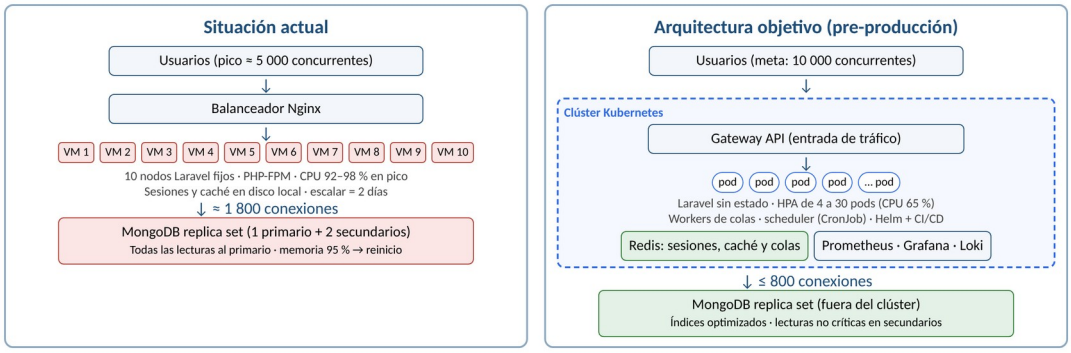



*Figura 1. Arquitectura actual frente a la arquitectura objetivo que construirá el equipo durante el Dojo.*

## 2. Ejercicio 1. Objetivos del DevOps Dojo

### 2.1. Objetivo general

Desarrollar, en 12 semanas, las competencias del equipo del portal para diagnosticar problemas de rendimiento, optimizar el acceso a MongoDB y operar la aplicación Laravel en Kubernetes con autoescalado, de forma que, al finalizar el programa, el entorno de pre-producción **soporte 10 000 usuarios concurrentes** (el doble del punto de quiebre actual) con una **latencia p95 inferior a 800 ms**, **menos del 1 % de errores** y **sin caídas del replica set**. Este objetivo responde directamente a la meta estratégica de la campaña de 2027.

### 2.2. Objetivos específicos

| Objetivo específico (SMART) | Indicador y meta | Plazo |
|---|---|---|
| **OE1. Diagnosticar con datos** el comportamiento del portal bajo carga. | Panel de observabilidad (Laravel, nodos y MongoDB) y prueba de carga que reproduce la caída a ≈ 5 000 usuarios. | Semana 2 |
| **OE2. Optimizar el acceso a MongoDB** con índices, caché y lecturas en secundarios. | Consultas p95 < 100 ms en las 20 más frecuentes; ≤ 800 conexiones al primario con 5 000 usuarios. | Semana 4 |
| **OE3. Contenerizar** la aplicación sin estado e integrarla en una canalización de CI. | Sesiones, caché y colas en Redis; imagen construida y probada en el 100 % de los cambios. | Semana 5 |
| **OE4. Desplegar en Kubernetes** con entrega continua. | Despliegue automatizado con Helm en ≤ 15 min, sin indisponibilidad; reversión en ≤ 5 min. | Semana 8 |
| **OE5. Escalar automáticamente** según la demanda. | HPA de 4 a 30 pods; nueva capacidad en < 3 min; 10 000 usuarios con p95 < 800 ms. | Semana 9 |
| **OE6. Asegurar el clúster** y la cadena de entrega. | RBAC mínimo, secretos cifrados, políticas de red y cero vulnerabilidades críticas. | Semana 10 |

| Objetivo específico (SMART) | Indicador y meta | Plazo |
|---|---|---|
| **OE7. Garantizar la resiliencia** ante fallos. | Ante la caída de un pod, un nodo o el primario de MongoDB, recuperación en < 1 min con errores < 1 %. | Semana 11 |
| **OE8. Desarrollar competencias y colaboración** entre áreas. | Evaluación técnica +25 %; asistencia ≥ 85 %; ≥ 80 % de acciones de mejora a tiempo. | Semanas 1-12 |

*Tabla 2. Objetivos específicos, indicadores y plazos.*

Los objetivos son alcanzables porque se trabaja sobre una sola aplicación, en pre-producción y con MongoDB fuera del clúster. La línea base se confirmará en las semanas 1 y 2.

## 3. Ejercicio 1. Beneficios esperados

| Beneficio | Justificación y contribución a la organización | Evidencia |
|---|---|---|
| **Escalabilidad bajo demanda** | El autoescalado añade pods en minutos en lugar de servidores en días, lo que permite atender la campaña de 2027 sin sobredimensionar la infraestructura todo el año. | Usuarios concurrentes soportados; tiempo de escalado. |
| **Disponibilidad y rendimiento** | Al reducir la carga y las conexiones a MongoDB se elimina la causa de las caídas, Kubernetes reinicia los pods que fallan y las páginas responden rápido en el pico. Protege los ingresos y la reputación. | Caídas al mes; errores 5xx; latencia p95. |
| **Uso eficiente de recursos** | Hoy los 10 servidores funcionan todo el día aunque el pico dure pocas horas; con el HPA se usan 4 pods en horas valle y se crece solo cuando hace falta. | CPU media; horas de cómputo consumidas. |
| **Despliegues sin interrupción** | Las actualizaciones progresivas de Kubernetes eliminan los 10 minutos de corte por despliegue y permiten publicar cambios con más frecuencia. | Tiempo de despliegue; indisponibilidad. |
| **Recuperación más rápida** | Alertas tempranas, procedimientos documentados y prácticas de caos preparan al equipo para resolver incidencias en minutos. | Tiempo de recuperación (MTTR). |
| **Competencias propias** | El equipo aprende Kubernetes, rendimiento y MongoDB haciendo, lo que reduce la dependencia de consultores externos. | Evaluación técnica inicial y final. |
| **Colaboración Dev-Ops-DBA** | El problema es compartido: las consultas son de Desarrollo, la configuración es del DBA y la capacidad es de Operaciones. Resolverlo juntos rompe los silos. | Encuesta de colaboración; acciones completadas. |

*Tabla 3. Beneficios esperados, justificación y evidencia.*

## 4. Ejercicio 2. Alcance del DevOps Dojo

### 4.1. Áreas y sistema incluidos

El Dojo abarca el **portal Laravel**, su **replica set de MongoDB** (solo optimización y monitoreo) y las cinco áreas que intervienen en su ciclo de vida: Desarrollo, Operaciones, Bases de datos, Calidad y Seguridad. Las prácticas se realizan en un **clúster Kubernetes de pre-producción** y en una copia del replica set con datos anonimizados.

### 4.2. Temas y competencias que se cubrirán

| Bloque temático | Contenidos | Competencia esperada |
|---|---|---|
| **Observabilidad y rendimiento** | Métricas, registros, paneles, pruebas de carga con k6 y punto de quiebre. | Diagnosticar cuellos de botella con datos. |
| **MongoDB** | Perfilador, *explain()*, índices, conexiones, preferencias de lectura y elecciones de primario. | Optimizar consultas y configurar el acceso al replica set. |
| **Laravel sin estado** | Sesiones, caché y colas en Redis; configuración por variables; ajuste de PHP-FPM. | Preparar la aplicación para escalar horizontalmente. |
| **Contenedores e integración continua** | Imagen multietapa, pruebas con PHPUnit, análisis estático y registro de imágenes. | Construir artefactos reproducibles y probados. |
| **Kubernetes** | Pods, Deployments, Services, Gateway API, ConfigMaps, Secrets, sondas, Jobs y CronJobs. | Desplegar y operar la aplicación en un clúster. |
| **Entrega continua** | Helm, despliegue progresivo y reversión. | Publicar cambios sin interrupción. |
| **Autoescalado** | *Requests* y *limits*, HPA, PodDisruptionBudget y planificación de capacidad. | Ajustar la capacidad a la demanda. |
| **DevSecOps** | RBAC, políticas de red, secretos, análisis de imágenes. | Aplicar seguridad en el clúster y la canalización. |
| **Resiliencia y cultura** | Pruebas de caos, procedimientos de incidencias, retrospectivas sin culpables. | Responder a fallos y aprender de ellos. |

*Tabla 4. Plan de temas y competencias del Dojo.*

### 4.3. Entregables y elementos fuera del alcance

**Entregables:** diagnóstico y línea base de carga; índices y configuración optimizada de MongoDB; aplicación sin estado en contenedores; canalización CI/CD con *chart* de Helm; despliegue en Kubernetes con autoescalado; controles de seguridad; panel y alertas; procedimientos de incidencias; e informe final con un **plan de migración a producción** y criterios de decisión.

**Fuera del alcance:** la migración a producción (se hará en una segunda fase con despliegue gradual), ejecutar MongoDB dentro de Kubernetes, fragmentar la base de datos (*sharding*), reescribir la aplicación e implantar varias regiones. Si tras la optimización el primario sigue por encima del 70 % de CPU, se recomendará evaluar su escalado en la segunda fase.

## 5. Ejercicio 2. Equipos de trabajo participantes

Participará un equipo multidisciplinario de **nueve personas** y tres roles de apoyo:

| Equipo o rol | N.º | Responsabilidades en el Dojo | Motivo de la selección | Beneficio de su formación |
|---|---|---|---|---|
| **Desarrollo Laravel** | 4 | Hacer la aplicación sin estado, optimizar consultas, crear pruebas y el *chart* de Helm. | Escriben las consultas que saturan MongoDB y generan la mayoría de los cambios. | Diseñar aplicaciones escalables y entender su impacto en producción. |

| Equipo o rol | N.° | Responsabilidades en el Dojo | Motivo de la selección | Beneficio de su formación |
|---|---|---|---|---|
| **Operaciones y plataforma** | 2 | Construir el clúster, la entrada de tráfico, el autoescalado y la observabilidad. | Administran hoy los 10 servidores y el balanceador Nginx. | Pasar de administrar servidores a operar una plataforma automatizada. |
| **Base de datos (DBA)** | 1 | Analizar consultas, crear índices y configurar el replica set. | MongoDB es el punto de fallo principal. | Trabajar con Desarrollo desde el diseño y no solo al resolver caídas. |
| **Calidad y rendimiento** | 1 | Diseñar las pruebas de carga y los criterios de aceptación. | Validará con datos si se cumple la meta de 10 000 usuarios. | Automatizar pruebas de rendimiento en la canalización. |
| **Seguridad** | 1 | Definir RBAC, políticas de red y gestión de secretos. | Kubernetes introduce nuevos riesgos de acceso. | Integrar la seguridad desde el inicio. |
| **Patrocinador (gerente de TI)** | 1 | Aprobar recursos y decidir la migración a producción. | Autoridad sobre presupuesto y prioridades. | Decidir con evidencias. |
| **Coordinador del Dojo** | 1 | Planificar sesiones, medir y gestionar la retroalimentación. | Líder técnico con experiencia en proyectos. | Capacidad para replicar el Dojo. |
| **Mentor Kubernetes (externo)** | 1 | Guiar las prácticas y revisar la arquitectura. | Aporta experiencia que el equipo aún no tiene. | Transfiere conocimiento al equipo. |

*Tabla 5. Equipos participantes, responsabilidades y justificación.*

### 5.1. Criterios de selección y coordinación

Se seleccionó a quienes tienen **relación directa** con el portal o sus incidencias, combinando **perfiles júnior y sénior**, con **disponibilidad** de 6 horas semanales acordada con cada jefe de área y **disposición para actuar como agentes de cambio** en la segunda fase. El equipo trabajará en tríos mixtos (desarrollo, operaciones y DBA o calidad), con una reunión de seguimiento de 15 minutos a mitad de semana y un tablero con responsable, fecha y criterio de aceptación de cada tarea. Los obstáculos que requieran recursos se escalarán al patrocinador.

## 6. Ejercicio 3. Cronograma, sesiones, recursos y tecnologías

### 6.1. Cronograma de sesiones

El Dojo se realizará del **12 de abril al 2 de julio de 2027**, con una sesión semanal los miércoles de 9:00 a 13:00 h (48 horas) y 2 horas semanales de práctica. La secuencia es deliberada: **primero medir, después optimizar y por último escalar** con el HPA (Kubernetes, s. f.), porque escalar una aplicación ineficiente solo multiplica las conexiones a MongoDB.

| Sem. | Fechas | Sesión y objetivo de aprendizaje | Actividad práctica | Entregable |
|---|---|---|---|---|
| 1 | 12-16 abr | **Observabilidad.** Medir el comportamiento del sistema. | Instalar Prometheus, Grafana, Loki y el exportador de MongoDB. | Panel inicial |

| Sem. | Fechas | Sesión y objetivo de aprendizaje | Actividad práctica | Entregable |
|---|---|---|---|---|
| 2 | 19-23 abr | **Pruebas de carga.** Encontrar el punto de quiebre. | Crear escenarios k6 y reproducir la caída con 5 000 usuarios. | Línea base |
| 3 | 26-30 abr | **MongoDB.** Optimizar consultas y conexiones. | Usar el perfilador y *explain()*, crear índices y configurar lecturas en secundarios. | Índices y configuración |
| 4 | 3-7 may | **Laravel sin estado.** Preparar la aplicación para escalar. | Mover sesiones, caché y colas a Redis; ajustar PHP-FPM; repetir la prueba. | Segunda medición |
| 5 | 10-14 may | **Contenedores y CI.** Construir artefactos probados. | Crear la imagen multietapa y la canalización con PHPUnit. | Imagen versionada |
| 6 | 17-21 may | **Fundamentos de Kubernetes.** Comprender sus objetos. | Desplegar el portal en un clúster local (kind). | Manifiestos base |
| 7 | 24-28 may | **Kubernetes en pre-producción.** Operar la aplicación completa. | Configurar Gateway API, workers, CronJob y migraciones como Job. | Portal en el clúster |
| 8 | 31 may-4 jun | **Helm y entrega continua.** Desplegar sin interrupción. | Crear el *chart* y automatizar el despliegue y la reversión. | Canalización CD |
| 9 | 7-11 jun | **Autoescalado.** Ajustar la capacidad a la demanda. | Definir *requests* y el HPA; prueba escalonada de 5 000 a 10 000 usuarios. | HPA validado |
| 10 | 14-18 jun | **DevSecOps.** Proteger el clúster. | RBAC, políticas de red hacia MongoDB, secretos y Trivy. | Informe de seguridad |
| 11 | 21-25 jun | **Resiliencia.** Responder a fallos. | Eliminar pods, drenar un nodo y forzar una elección de primario en MongoDB. | Procedimientos de incidencias |
| 12 | 28 jun-2 jul | **Evaluación y cierre.** Demostrar los resultados. | Prueba final con 10 000 usuarios, comparación y presentación. | Informe y plan de migración |

*Tabla 6. Cronograma de sesiones del DevOps Dojo.*

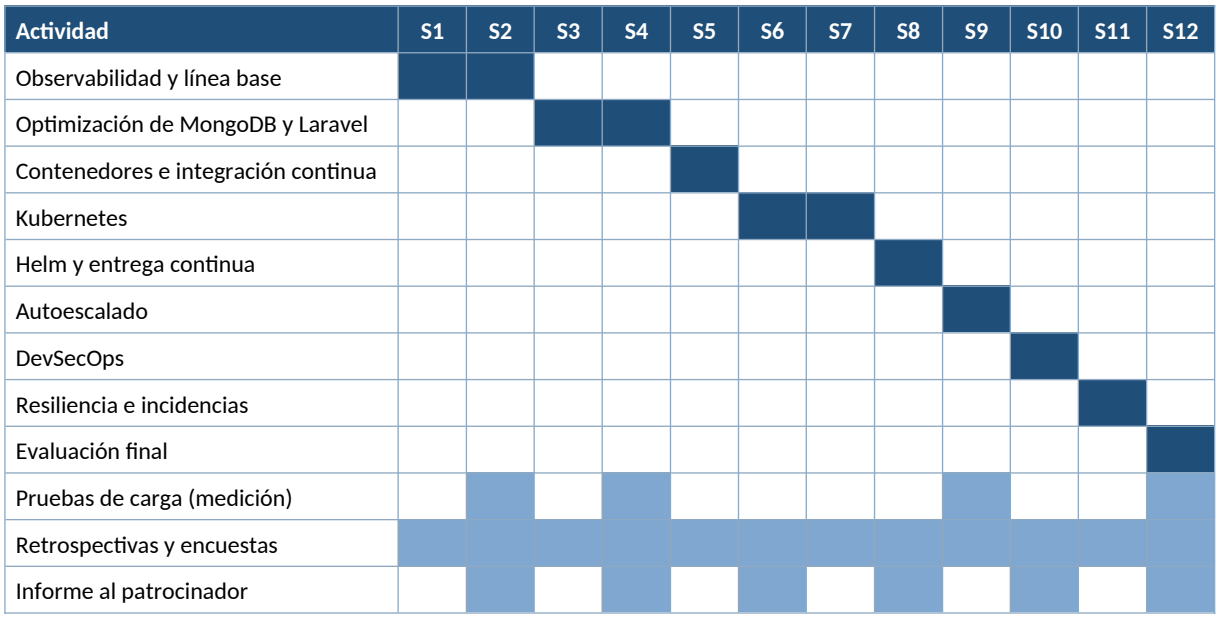



*Figura 2. Diagrama de Gantt del DevOps Dojo.*

### 6.2. Estructura de cada sesión

Cada sesión sigue la secuencia: **teoría** (30 min), **demostración** (30 min), **ejercicio práctico** en tríos (120 min), **revisión de entregables** (30 min) y **retrospectiva con encuesta** (30 min).

### 6.3. Recursos y tecnologías

| Tipo | Recursos necesarios |
|---|---|
| **Humanos** | Coordinador, mentor externo (48 h de sesión y 24 h de apoyo), 9 participantes con 6 h semanales y patrocinador. |
| **Infraestructura** | Clúster Kubernetes de pre-producción (3 nodos de control y 6 de trabajo de 8 vCPU y 16 GB), copia del replica set de MongoDB con datos anonimizados, servidor Redis y 2 máquinas generadoras de carga. |
| **Físicos y documentales** | Sala con proyector; portátiles con Docker y kind; guías de laboratorio, criterios de aceptación y plantillas de retrospectiva y de incidencias. |

*Tabla 7. Recursos del Dojo.*

| Área | Tecnología propuesta | Uso en el Dojo |
|---|---|---|
| **Código y CI/CD** | GitLab y GitLab CI/CD | Repositorio, revisiones, canalizaciones y registro de imágenes. |
| **Calidad** | PHPUnit y Larastan | Pruebas automatizadas y análisis estático de Laravel. |
| **Contenedores** | Docker (kind para prácticas locales) | Imagen PHP-FPM y Nginx reproducible. |
| **Orquestación** | Kubernetes (RKE2), Helm y Gateway API | Despliegue, entrada de tráfico y versionado de la configuración. |
| **Escalado** | HPA y metrics-server | Ajuste automático del número de pods según la CPU. |
| **Datos** | MongoDB 7 (replica set) y Redis | Base de datos optimizada; sesiones, caché y colas. |
| **Pruebas de carga** | k6 | Simulación de 5 000 a 10 000 usuarios concurrentes. |
| **Observabilidad** | Prometheus, Grafana, Loki y MongoDB Exporter | Métricas, registros, paneles y alertas. |
| **Seguridad** | Trivy, Sealed Secrets y NetworkPolicies | Vulnerabilidades, secretos cifrados y aislamiento de red. |

*Tabla 8. Tecnologías del Dojo.*

## 7. Ejercicio 4. Métricas de desempeño

Las métricas combinan indicadores de **capacidad y confiabilidad** del portal, las **métricas DORA** de entrega de software (Forsgren et al., 2018) y los indicadores de **aprendizaje**. Todas las pruebas de carga se ejecutan con el mismo escenario k6 para que los resultados sean comparables:

| Indicador (objetivo) | Fórmula o método | Base | Meta | Fuente | Frec. |
|---|---|---|---|---|---|
| Usuarios concurrentes soportados (OG, OE5) | Máximo de usuarios con p95 < 800 ms y errores < 1 % en la prueba escalonada. | ≈ 5 000 | ≥ 10 000 | k6 | S2, S4, S9, S12 |
| Latencia p95 en el pico (OG) | Percentil 95 del tiempo de respuesta con 5 000 usuarios. | 1 900 ms | < 500 ms | k6 y Grafana | Por prueba |
| Tasa de errores 5xx (OG) | Respuestas 5xx ÷ total de respuestas × 100. | 12 % | < 1 % | Gateway y Prometheus | Por prueba |
| Conexiones al primario (OE2) | Máximo de conexiones activas con 5 000 usuarios. | ≈ 1 800 | ≤ 800 | MongoDB Exporter | Por prueba |
| Tiempo de escalado (OE5) | Tiempo desde que se supera el 65 % de CPU hasta que los nuevos pods reciben tráfico. | 2 días | < 3 min | Prometheus | Por prueba |
| Tiempo de despliegue (OE4) | Mediana desde el inicio del despliegue hasta superar las sondas de salud. | 120 min | ≤ 15 min | GitLab CI/CD | Semanal |
| Tasa de fallos de cambios (OE4) | Despliegues que requieren reversión ÷ total × 100. | 20 % | ≤ 10 % | GitLab | Semanal |
| Tiempo de recuperación (OE7) | Mediana desde el fallo inyectado hasta el servicio normal. | 95 min | < 1 min | Grafana | S11 y S12 |
| Vulnerabilidades críticas (OE6) | Vulnerabilidades críticas abiertas en imágenes. | Sin medir | 0 | Trivy | Por ejecución |
| Competencias y participación (OE8) | Prueba práctica final frente a inicial; asistencia; acciones a tiempo. | S1 | +25 %; ≥ 85 %; ≥ 80 % | Evaluación y tablero | S1, S12 y semanal |

*Tabla 9. Métricas de desempeño del DevOps Dojo.*

### 7.1. Método de medición y análisis

Los datos se extraerán automáticamente de k6, Prometheus, GitLab y Trivy hacia un panel compartido de Grafana; en la semana 1 se documentará la fórmula, la fuente y el responsable de cada indicador. El análisis se hará en tres niveles:

- **Operativo (semanal):** el equipo revisa los resultados de las pruebas y los fallos de la canalización en la retrospectiva.
- **Táctico (quincenal):** el coordinador compara cada prueba de carga con la anterior; si un indicador no mejora, se analiza la causa antes de avanzar al siguiente bloque.
- **Estratégico (semana 12):** el patrocinador decide la migración a producción si se cumplen las metas de usuarios, latencia, errores y recuperación.

## 8. Ejercicio 4. Sistema de retroalimentación

El sistema sigue un ciclo de cinco pasos que se repite al terminar cada sesión:

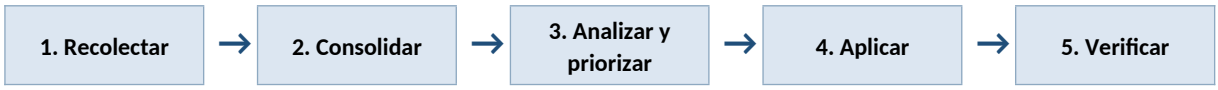



*Figura 3. Ciclo de retroalimentación aplicado al término de cada sesión.*

### 8.1. Recolección

Al cierre de cada sesión, los participantes responden una **encuesta anónima de 3 minutos** con una escala del 1 (muy en desacuerdo) al 5 (muy de acuerdo):

| N.° | Pregunta | Dimensión |
|---|---|---|
| P1 | Los objetivos de aprendizaje de la sesión fueron claros. | Contenido |
| P2 | El ejercicio me ayudó a entender el problema de escalabilidad del portal. | Relevancia |
| P3 | La dificultad del ejercicio fue adecuada. | Dificultad |
| P4 | Me siento capaz de aplicar lo aprendido sin ayuda. | Confianza |
| P5 | El clúster y las herramientas funcionaron correctamente. | Recursos |
| P6 | (Abierta) ¿Qué cambiarías para la próxima sesión? | Mejora |

*Tabla 10. Encuesta de cierre de sesión.*

Se complementa con la **retrospectiva** (¿qué funcionó?, ¿qué nos dificultó? y ¿qué cambiamos?), el registro de obstáculos del **tablero** y la **revisión de entregables**.

### 8.2. Análisis y reglas de ajuste

El coordinador consolida los resultados en 24 horas, calcula la media de cada pregunta y agrupa los comentarios por categoría. Se aplican estas reglas:

- Media **menor de 3,5** en P1 o P3: el mentor rediseña el ejercicio y lo repasa al inicio de la siguiente sesión.
- Media **menor de 3,5** en P4: se programa una práctica de refuerzo de una hora en la semana.
- Media **menor de 4,0** en P5: Operaciones corrige el entorno antes de la siguiente sesión.
- Si una prueba de carga **no mejora** respecto a la anterior: se dedica la primera hora de la siguiente sesión a analizar la causa en equipo.

### 8.3. Aplicación, seguimiento y evaluación

Cada retrospectiva termina con **un máximo de tres acciones** (problema, acción, responsable, fecha y evidencia), que se verifican al inicio de la sesión siguiente. El patrocinador recibe un informe quincenal. El sistema será eficaz si el 80 % responde cada encuesta, el 80 % de las acciones se completa a tiempo y la media de la encuesta aumenta a lo largo del programa.

## 9. Conclusiones

Organizar el Dojo alrededor de un reto real, la caída del portal por encima de los 5 000 usuarios, hace que cada sesión produzca un avance medible sobre un problema del negocio. La propuesta parte de una idea central: **Kubernetes por sí solo no resuelve el problema**. Si se escalara la aplicación sin optimizar MongoDB, el aumento de pods multiplicaría las conexiones y la base de datos caería antes. Por eso el cronograma avanza de forma progresiva: medir, optimizar, contenerizar, orquestar, escalar y comprobar la resiliencia, con objetivos SMART, un equipo multidisciplinario que incluye al DBA y un alcance delimitado a pre-producción.

Las métricas permitirán demostrar con datos si se alcanzan los 10 000 usuarios, y la retroalimentación corregirá el programa cada semana. Se recomienda aprobar la migración a producción solo si se cumplen las metas, y realizarla de forma gradual.

## 10. Referencias bibliográficas


DORA. (s. f.). *How to transform your organization*. https://dora.dev/guides/how-to-transform/

Forsgren, N., Humble, J. y Kim, G. (2018). *Accelerate: The science of lean software and DevOps. Building and scaling high performing technology organizations*. IT Revolution Press.

Kim, G., Humble, J., Debois, P. y Willis, J. (2016). *The DevOps handbook: How to create world-class agility, reliability, and security in technology organizations*. IT Revolution Press.

Kubernetes. (s. f.). *Horizontal Pod Autoscaling*. The Kubernetes Authors. https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/

Microsoft. (s. f.). *Introduce DevOps Dojo: Create efficiencies that support your business* [Ruta de aprendizaje]. Microsoft Learn. https://learn.microsoft.com/en-us/training/paths/devops-dojo-white-belt-foundation/

Microsoft. (2021). *Intro of DevOps Dojo*. Azure DevOps Blog. https://devblogs.microsoft.com/devops/intro-of-devops-dojo/

MongoDB. (s. f.). *Replication*. MongoDB Manual. https://www.mongodb.com/docs/manual/replication/